

100 - Custom authorization requirements and handlers

Simple requirements:

```
options.AddPolicy("EditRolePolicy",  
    policy => policy.RequireClaim("Edit Role"));  
  
options.AddPolicy("EditRolePolicy",  
    policy => policy.RequireClaim("Edit Role", "true"));  
  
options.AddPolicy("EditRolePolicy",  
    policy => policy.RequireClaim("Edit Role", "true", "yes"));
```

Multiple requirements:

Policies with multiple built-in requirements

For this policy to succeed, the user must be in the **Admin** role **AND** must have **Edit Role** claim type with a claim value of either **true** or **yes**

```
options.AddPolicy("EditRolePolicy", policy => policy  
    .RequireClaim("Edit Role", "true", "yes")  
    AND .RequireRole("Admin"));
```

OR

The user must be in the **Admin** role and has claim type **Edit Role** with a value of **true**
OR
The user must be in the **Super Admin** role

```
options.AddPolicy("EditRolePolicy", policy => policy.RequireAssertion(context =>  
context.User.IsInRole("Admin") &&  
context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true")  
context.User.IsInRole("Super Admin")));
```

OR

От Intellysense се вижда, че Build-in Requirements са съответно:

.RequireClaim - .ClaimsAuthorizationRequirement

```
.RequireClaim("Delete Role"));
```

AuthorizationPolicyBuilder AuthorizationPolicyBuilder.RequireClaim(string claimType) (+ 2 overloads)
Adds a Microsoft.AspNetCore.Authorization.Infrastructure.ClaimsAuthorizationRequirement to the current

.RequireRole - .RolesAuthorizationRequirement

```
.RequireRole("Admin"));
```

AuthorizationPolicyBuilder AuthorizationPolicyBuilder.RequireRole(params string[] roles) (+ 1 overload)
Adds a Microsoft.AspNetCore.Authorization.Infrastructure.RolesAuthorizationRequirement to the current in

.RequireAssertion - .AssertionRequirement

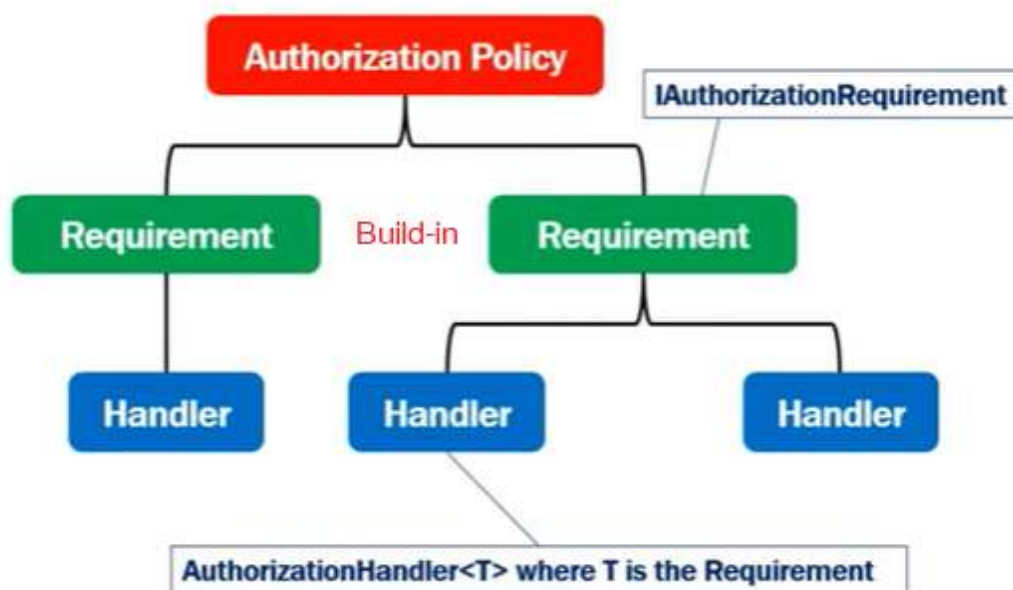
```
RequireAssertion(context =>
    AuthorizationPolicyBuilder.RequireAssertion(System.Func<AuthorizationHandlerContext, bool>
    handler) (+ 1 overload)
    Adds an Microsoft.AspNetCore.Authorization.Infrastructure.AssertionRequirement to the current instance.
```

Built-in asp.net core authorization requirements

- [RequireClaim\(\)](#) method adds [ClaimsAuthorizationRequirement](#).
- [RequireRole\(\)](#) method adds [RolesAuthorizationRequirement](#).
- [RequireAssertion\(\)](#) method adds [AssertionRequirement](#).

За прости приложения, тези Requirements ще свършат работа, но за по-сложни, трябва да въведем персонализирани:

Custom authorization requirement



An authorization policy has one or more requirements. Each authorization requirement has one or more handlers.

All the built-in authorization requirements implement [IAuthorizationRequirement](#) interface. So to create a custom authorization requirement, we need to implement [IAuthorizationRequirement](#) interface. This is an empty marker interface, which means there is nothing in this interface that our custom requirement class must implement.

It is in the [authorization handler](#) that we write our logic to allow or deny access to a resource like a [controller action](#) for example. An authorization handler implements [AuthorizationHandler<T>](#) where **T** is the type of requirement.

For example, let's say, an **admin** user can assign or remove roles of other admin users but not his own roles. To achieve this, we need to know the logged-in **UserID** and the **UserId** of

the **Admin** being edited. If they are the same we do not want to allow access. The admin **UserID** being edited is passed in the URL as a query string parameter. From the authorization handler we will have access to the route data and URL query string parameters. Dependency injection is also supported. This means, we can even inject and use other services if required.